Spring boot

Spring Data JPA



hamdi.aloulou@isima.u-monastir.tn

Spring Data JPA

- L'application a besoin de communiquer avec une base de données:
  - Créer des objets, Chercher des objets, Supprimer des objets, etc.
- Normalement, nous devrions implémenter le code qui exécute ces actions
  - Un code qui soit capable d'exécuter des requêtes SQL
- Le module Spring Data JPA nous évite cette tâche !
  - Avec Spring Data JPA nous écrivons uniquement des interfaces, et c'est lui qui nous fournira l'implémentation de ces dernières

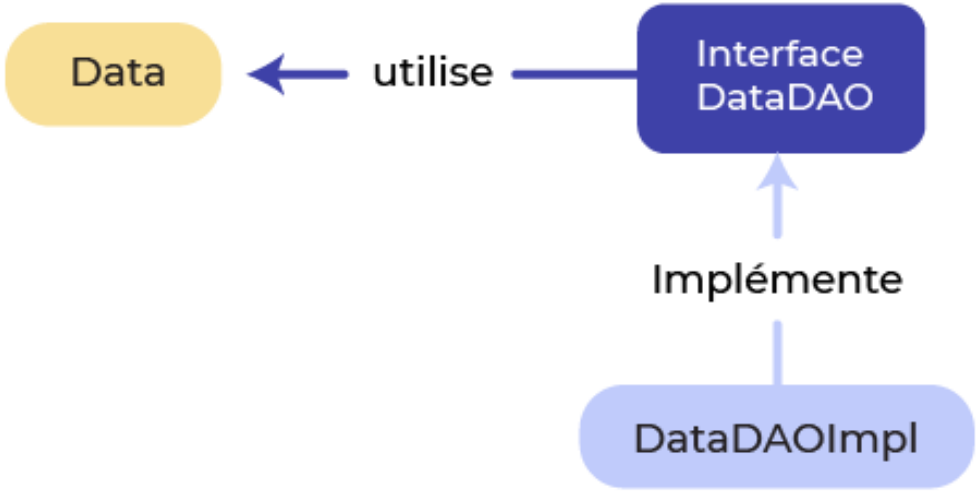
Spring Data JPA

- Le module Spring Data facilite grandement l'accès aux données.
- Génère toutes sortes d'opérations vers la base de données
  - sans écrire la moindre requête.
  - sans implémentation DAO.

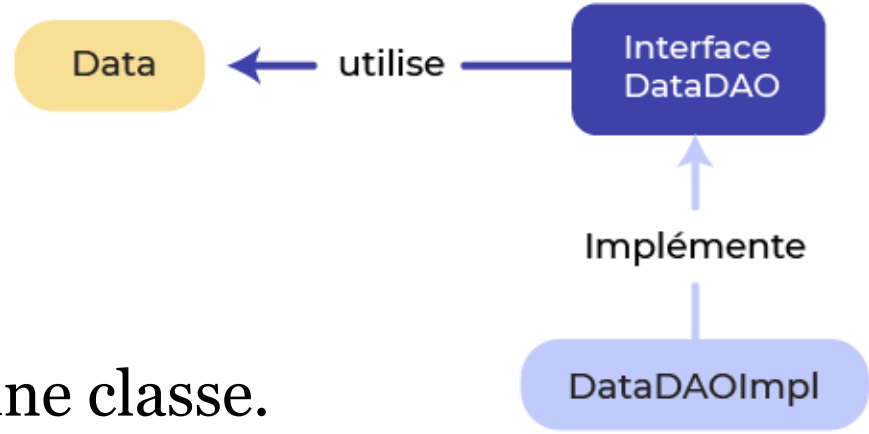
Spring Data JPA – Patron DAO

› Le module Spring Data JPA met en pratique le design pattern DAO : Data Access Object.

› Le patron de conception DAO décrit la communication avec la source de données via la structure suivante :



Spring Data JPA – Principe du Patron DAO



› Dans le patron DAO :

- La donnée (Data) est modélisée via une classe.
- Les opérations sur la base de données sont spécifiées via une interface (DataDAO).
- Les opérations sont implémentées dans une classe (DataDAOImpl).

Spring Data JPA – Patron DAO

- Spring Data JPA nous demande de respecter ce principe :
  - Implémenter des classes qui représentent nos données : Les entités.
  - Créer des interfaces pour manipuler ces entités : Les repositories.
- Spring Data JPA demande de ne pas implémenter les opérations :
  - C'est la classe la plus longue et la plus complexe à implémenter.
  - **C'est lui qui s'en charge de cette partie**

Spring Data JPA - Intégration

› **Le starter** `spring-boot-starter-data-jpa` **va permettre d'importer les dépendances nécessaires**

```
<dependency>
	<groupId>org.springframework.boot</groupId>
	<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

Spring Data JPA - Intégration

- Une classe contient **les attributs** qui décrivent l'objet manipulé par l'application avec **les Getters et Setters**
- L'annotation **@Entity** est appliquée à cette classe afin d'indiquer qu'il s'agit d'une classe qui décrit un objet.
- L'annotation **@Id** est appliquée sur l'attribut identifiant de l'objet.

```
@Entity
public class Product
{
    @Id
    private int id;
    …
}
```

Spring Data JPA - Intégration

- Dans la couche DAO, on définit une interface pour chaque entité de la couche métier (Repository)
- Cette interface décrit les opérations sur l'entité.
- Elle doit **hériter de l'interface *JPArepository*.**
- L'annotation **@Repository** est appliquée à l'interface afin d'indiquer qu'il s'agit d'une classe qui gère les données.

```
@Repository
public interface ProdcutDAO extends JpaRepository<Product, Integer>
{ }
```

Nom de l'entité qu'on manipule

Type de l'identifiant de l'entité

Spring Data JPA - Intégration

- La classe Controlleur de l'API REST assure la communication avec les autres services web.
- On applique l'annotation ***@RestController*** *à cette classe.*
- Elle reçoit des demandes à travers des URIs envoyées par les clients et fait appel aux méthodes définis dans le Repository.

```
@RestController
public class ProductController {

    private final ProductDao productDao;

    @GetMapping("/Produits")
    public List<Product> listeProduits() {
        …
    }

}
```

Spring JPA - principes

- L'entité du model est annotée obligatoirement avec l'annotation ***@Entity***
- La communication avec la base de données s'effectue via une interface annotée avec ***@Repository***.
- La communication avec le service web s'effectue en utilisant des URIs à travers une classe annotée avec ***@RestController*** qui définit le code à exécuter quand ces URIs sont reçus.

Accès aux données – Les noms des méthodes

- Dans Spring Data JPA, les noms des méthodes ne sont pas choisis au hasard.
- Suivre **les conventions de Spring Data JPA** permet de bénéficier de certaines fonctionnalités qui font gagner beaucoup de temps.

Accès aux données – Les noms des méthodes CRUD (1/8)

› Méthodes de creation :

All Methods | Instance Methods | Abstract Methods

Modifier and Type	Method and Description
long	**count**() Returns the number of entities available.
void	**delete**(T entity) Deletes a given entity.
void	**deleteAll**() Deletes all entities managed by the repository.
void	**deleteAll**(Iterable<? extends T> entities) Deletes the given entities.
void	**deleteById**(ID id) Deletes the entity with the given id.
boolean	**existsById**(ID id) Returns whether an entity with the given id exists.
Iterable<T>	**findAll**() Returns all instances of the type.
Iterable<T>	**findAllById**(Iterable<ID> ids) Returns all instances of the type with the given IDs.
Optional<T>	**findById**(ID id) Retrieves an entity by its id.
<S extends T> S	**save**(S entity) Saves a given entity.
<S extends T> Iterable<S>	**saveAll**(Iterable<S> entities) Saves all given entities.

Accès aux données – Les noms des méthodes CRUD (2/8)

› Méthodes de creation :

- **save :** enregistre une entité et retourne l'entité sauvegardée (avec une valeur de l'id)
- **saveAll :** enregistre un ensemble d'entités et retourne les entités sauvegardées (avec leurs valeurs d'id)

Accès aux données – Les noms des méthodes CRUD (3/8)

› Méthodes de lecture :

All Methods | Instance Methods | Abstract Methods

Modifier and Type	Method and Description
long	**count**() Returns the number of entities available.
void	**delete**(T entity) Deletes a given entity.
void	**deleteAll**() Deletes all entities managed by the repository.
void	**deleteAll**(Iterable<? extends T> entities) Deletes the given entities.
void	**deleteById**(ID id) Deletes the entity with the given id.
boolean	**existsById**(ID id) Returns whether an entity with the given id exists.
Iterable<T>	**findAll**() Returns all instances of the type.
Iterable<T>	**findAllById**(Iterable<ID> ids) Returns all instances of the type with the given IDs.
Optional<T>	**findById**(ID id) Retrieves an entity by its id.
<S extends T> S	**save**(S entity) Saves a given entity.
<S extends T> Iterable<S>	**saveAll**(Iterable<S> entities) Saves all given entities.

Accès aux données – Les noms des méthodes CRUD (4/8)

› Méthodes de lecture :

- **findAll :** permet de récupérer, sous forme d'un objet List, toutes les lignes de la table associée
- **findAllById :** permet de récupérer un ensemble de lignes à partir d'une liste d'id
- **findById :** permet de récupérer une ligne de la table à partir de l'id

Accès aux données – Les noms des méthodes CRUD (5/8)

› Méthodes de mise à jour :

All Methods | Instance Methods | Abstract Methods

Modifier and Type	Method and Description
long	**count**() Returns the number of entities available.
void	**delete**(T entity) Deletes a given entity.
void	**deleteAll**() Deletes all entities managed by the repository.
void	**deleteAll**(Iterable<? extends T> entities) Deletes the given entities.
void	**deleteById**(ID id) Deletes the entity with the given id.
boolean	**existsById**(ID id) Returns whether an entity with the given id exists.
Iterable<T>	**findAll**() Returns all instances of the type.
Iterable<T>	**findAllById**(Iterable<ID> ids) Returns all instances of the type with the given IDs.
Optional<T>	**findById**(ID id) Retrieves an entity by its id.
<S extends T> S	**save**(S entity) Saves a given entity.
<S extends T> Iterable<S>	**saveAll**(Iterable<S> entities) Saves all given entities.

Accès aux données – Les noms des méthodes CRUD (6/8)

› Méthodes de mise à jour : **Gérée par les mêmes méthodes de création**

  – **save(S entity)**

  – **saveAll(Iterable<S> entities)**

› Si l'objet existe dans la table, alors il s'agira d'une requête update et non pas insert. **(En utilisant son identifiant unique)**

Accès aux données – Les noms des méthodes CRUD (7/8)

› Méthodes de mise à jour (Exemple)

```
Optional<Product> existingProduct = productdao.findById(1);
Product pro1=existingProduct.get();

System.out.println("***Le nom actuel du produit 1 est: " + pro1.getNom());

pro1.setNom("Telephone");
productdao.save(pro1);

pro1 = productdao.findById(1).get();
System.out.println("***Le nom, après modification, du produit 1 est: " +
pro1.getNom());
```

Accès aux données – Les noms des méthodes CRUD (8/8)

› Méthodes de suppression :

All Methods | Instance Methods | Abstract Methods

Modifier and Type	Method and Description
long	**count**() Returns the number of entities available.
void	**delete**(T entity) Deletes a given entity.
void	**deleteAll**() Deletes all entities managed by the repository.
void	**deleteAll**(Iterable<? extends T> entities) Deletes the given entities.
void	**deleteById**(ID id) Deletes the entity with the given id.
boolean	**existsById**(ID id) Returns whether an entity with the given id exists.
Iterable<T>	**findAll**() Returns all instances of the type.
Iterable<T>	**findAllById**(Iterable<ID> ids) Returns all instances of the type with the given IDs.
Optional<T>	**findById**(ID id) Retrieves an entity by its id.
<S extends T> S	**save**(S entity) Saves a given entity.
<S extends T> Iterable<S>	**saveAll**(Iterable<S> entities) Saves all given entities.

Accès aux données – Les noms des méthodes dérivées – Conventions (1/5)

› Il faut utilisé **les noms des attributs** définis **dans l'entité** et **les conventions de Spring Data JPA** pour nommer ces méthodes.

› Spring Data permet de générer la requête SQL automatiquement **en partant du nom de votre méthode !**

Accès aux données – Les noms des méthodes dérivées – Conventions (2/5)

Keyword	Sample	JPQL snippet
`Distinct`	`findDistinctByLastnameAndFirstname`	`select distinct … where x.lastname = ?1 and x.firstname = ?2`
`And`	`findByLastnameAndFirstname`	`… where x.lastname = ?1 and x.firstname = ?2`
`Or`	`findByLastnameOrFirstname`	`… where x.lastname = ?1 or x.firstname = ?2`
`Is`, `Equals`	`findByFirstname`, `findByFirstnameIs`, `findByFirstnameEquals`	`… where x.firstname = ?1`
`Between`	`findByStartDateBetween`	`… where x.startDate between ?1 and ?2`
`LessThan`	`findByAgeLessThan`	`… where x.age < ?1`
`LessThanEqual`	`findByAgeLessThanEqual`	`… where x.age <= ?1`
`GreaterThan`	`findByAgeGreaterThan`	`… where x.age > ?1`
`GreaterThanEqual`	`findByAgeGreaterThanEqual`	`… where x.age >= ?1`
`After`	`findByStartDateAfter`	`… where x.startDate > ?1`
`Before`	`findByStartDateBefore`	`… where x.startDate < ?1`

Accès aux données – Les noms des méthodes dérivées – Conventions (3/5)

Keyword	Sample	JPQL snippet
`IsNull`, `Null`	`findByAge(Is)Null`	`… where x.age is null`
`IsNotNull`, `NotNull`	`findByAge(Is)NotNull`	`… where x.age not null`
`Like`	`findByFirstnameLike`	`… where x.firstname like ?1`
`NotLike`	`findByFirstnameNotLike`	`… where x.firstname not like ?1`
`StartingWith`	`findByFirstnameStartingWith`	`… where x.firstname like ?1` (parameter bound with appended `%`)
`EndingWith`	`findByFirstnameEndingWith`	`… where x.firstname like ?1` (parameter bound with prepended `%`)
`Containing`	`findByFirstnameContaining`	`… where x.firstname like ?1` (parameter bound wrapped in `%`)
`OrderBy`	`findByAgeOrderByLastnameDesc`	`… where x.age = ?1 order by x.lastname desc`

Accès aux données – Les noms des méthodes dérivées – Conventions (4/5)

Keyword	Sample	JPQL snippet
`Not`	`findByLastnameNot`	`… where x.lastname <> ?1`
`In`	`findByAgeIn(Collection<Age> ages)`	`… where x.age in ?1`
`NotIn`	`findByAgeNotIn(Collection<Age> ages)`	`… where x.age not in ?1`
`True`	`findByActiveTrue()`	`… where x.active = true`
`False`	`findByActiveFalse()`	`… where x.active = false`
`IgnoreCase`	`findByFirstnameIgnoreCase`	`… where UPPER(x.firstname) = UPPER(?1)`

Accès aux données – Les noms des méthodes dérivées – Conventions (5/5)

- Exemple : Récupérer des produits dont le prix est supérieur à 400.
  - Le nom de la méthode doit être: **findByPrixGreaterThan(int prixLimit)**
  - Toute la logique est fournie par le nom de la méthode !
    - ***FindBy* :** indique que l'opération à exécuter est un SELECT.
    - ***Prix* :** fournit le nom de la propriété sur laquelle le SELECT s'applique.
    - ***GreaterThan* :** définit une condition "plus grand que".
    - ***prixLimit :*** Indique la valeur à appliquer à la condition.
  - Cette méthode génère la requête SQL suivante :
    - select * from product where prix > [prixLimit]

Les requêtes JPQL et SQL

- En cas où, on ne peut pas exprimer notre besoin fonctionnel à l'aide d'une requête CRUD ou une requête dérivée, Spring Data JPA nous permet d'écrire:
  - Une requête JPQL (Utilisant les entités) ou
  - Une requête native SQL (Utilisant directement les tables)
- Il faut, dans l'interface Repository:
  - Ajouter l'annotation @Query et donner la requête avec le langage JPQL ou SQL
  - Décrire la méthode en spécifiant les types des paramètres

Le langage JPQL

- Le langage JPQL (Java Persistence Query Language) permet de décrire ce que l'application recherche
  - JPQL est un langage de requêtes adaptée à la spécification JPA
  - Inspiré du langage SQL mais adapté aux entités JPA
- Permet de manipuler les entités JPA et pas les tables d'une base de données
- Supporte des requêtes de type **select, update et delete**

Le langage JPQL

- Les requêtes JPQL manipule des entités et non pas des tables.
- Les identificateurs désignent les entités et leurs propriétés et pas les tables et leurs colonnes
- Les seules classes qui peuvent être explicitement désignées dans une requête (clause from) sont les entités (@Entity)
- Le nom d'une entité est donné par l'attribut name de @Entity
  - Par défaut c'est le nom de la classe

Le langage JPQL : Les clauses d'un select

- **select :** type des objets ou valeurs renvoyées
- **from :** où les données sont récupérées
- **where :** sélectionne les données
- **group by :** regroupe des données
- **having :** sélectionne les groupes
- **order by :** ordonne les données

Le langage JPQL : Les clauses Where et Having

› Ces clauses peuvent comporter les mots-clés suivants :

- [NOT] LIKE, [NOT] BETWEEN, [NOT] IN
- AND, OR, NOT
- [NOT] EXISTS
- ALL, SOME/ANY
- IS [NOT] NULL
- IS [NOT] EMPTY, [NOT] MEMBER OF (pour les collections)

Le langage JPQL : Exemples

- Sélectionner toutes les données des Employés
  - select e from Employe as e
  - select e from Employe e
  - On sélectionne des objets e de type Employe
- Sélectionner les noms et salaires des employés
  - select e.nom, e.salaire from Employe e

Le langage JPQL : Les requêtes paramétrées

- Un paramètre peut être désigné
  - par son numéro (?n), par exemple ?1, ?2, ?3…..
  - par son nom (:nom) par exemple :titre, :dateNai,…
- Les paramètres sont numérotés à partir de 1
- Un paramètre peut être utilisé plus d'une fois dans une requête
- L'usage des paramètres nommés est recommandé (plus lisible)

Le langage JPQL : Les requêtes paramétrées

- Attention
  - le même nombre de paramètre dans la requête et dans la méthode
  - Respecter l'ordre et le type des paramètres
- Paramètre numéroté :

```
import org.springframework.data.jpa.repository.Query;
public interface EtudiantDAO extends JpaRepository<Etudiant, Long>
{
        @Query("SELECT i FROM Etudiant i WHERE naissance <= ?1")
        List<Etudiant> findNaissanceSince(Date date);
}
```

- Paramètre nommé :

```
@Query("SELECT i FROM Etudiant i WHERE naissance <= :date")
List<Etudiant> findNaissanceSince(Date date);
```